Network Knight System Design

Design phase report

AI Assisted Cybersecurity Log Monitoring and Threat Detection System

Arif Shahriar Anik | Senior Design | October 28 2026

Our primary design connects selected security records to persistent evidence and explainable alerts. The current local baseline uses a Windows collector, Node.js, SQLite and a browser dashboard. AI assistance and broader operating system support remain development goals.

Design status: Existing describes implemented code; planned describes work to build and verify. The initial demo screenshots illustrate the interface and may change as development progresses.

Assignment coverage

Professor requirement	Location in this report
System Architecture	Section 1 and Figure 1
Database or Data Model Design	Section 2 and Figures 2 and 3
Technology Stack and Design Justification	Section 3
Standards Cybersecurity Frameworks Benchmarks and Constraints	Section 4
Context Diagram	Section 5 and Figure 4
Data Flow Diagram	Section 6 and Figure 5
User Interface and Experience Design	Section 7 and Figures 6 through 10
Security and Risk Considerations	Section 8 and Figure 11

Supporting material: Appendix A records the implementation baseline and original measurable requirements. Appendix B gives the local demonstration steps. Appendix C provides the team explanation guide. Appendix D preserves the implementation checklist.

1 System Architecture

Network Knight monitors selected security events on a Windows workstation. It collects Windows Event Logs and requests to its own monitored HTTP endpoints, normalizes and stores records, evaluates six explainable rules, and presents alerts with supporting evidence. An operator investigates alerts and acknowledges, resolves, or reopens them.

The architecture combines a browser client, modular monolithic Node.js backend, separate PowerShell collector, and SQLite database. Presentation, application logic, and persistence are three logical tiers deployed on one workstation. This supports an inexpensive academic prototype without a cloud dependency.

Status convention: Existing means present in the inspected code. Planned means a proposed implementation or operational improvement. Benchmark targets are acceptance criteria, not measured results. No framework certification or full compliance is claimed.

The supported deployment assumes one trusted Windows operator and an uncompromised host. The tool does not capture arbitrary network traffic, monitor every application, use AI, or automatically block threats. Its rejecting lab login endpoint demonstrates detection and is not dashboard authentication.

Validation and detection are logical modules inside the backend rather than separate services. All HTTP traffic remains on loopback.

Component	Responsibility	Access boundary
Browser	Presentation, filters, evidence and investigation actions	Untrusted log content must be escaped
HTTP server	Static assets, routes and API controls	Local binding; dashboard currently trusts OS user
Ingestion logic	Authenticate collector, validate batch, normalize, deduplicate and commit	Bearer key for ingestion and heartbeat
Rule engine	Correlation, alert creation and evidence updates	Server side business logic
SQLite	Persistent events, alerts and collector health	Local file; no database listener
PowerShell collector	Channel queries, XML parsing, upload and heartbeat	Security channel may need elevated rights
Runtime files	Secret key and acknowledged record positions	Windows ACL protection required

System architecture and component responsibilities

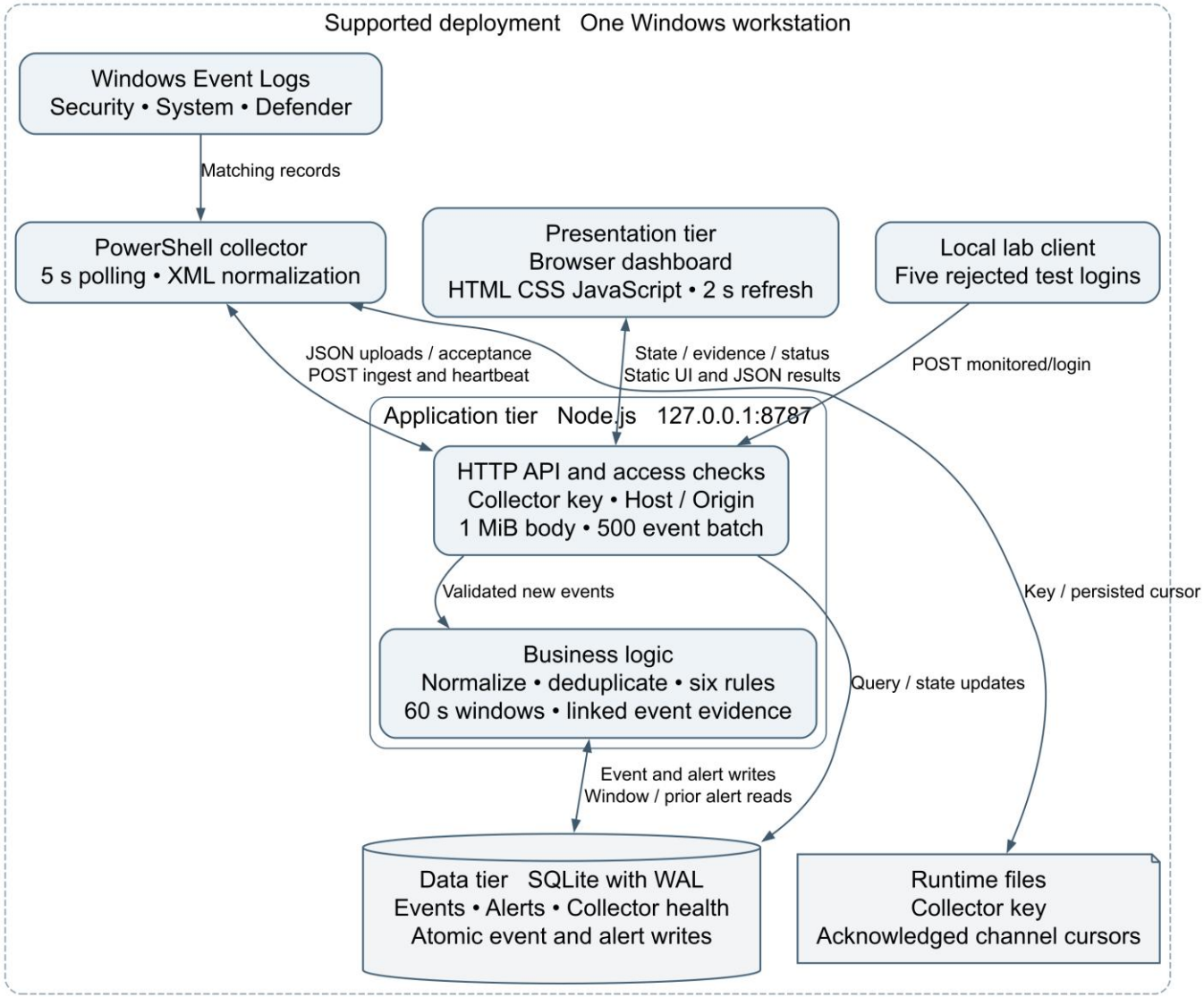


Figure 1 System architecture and component responsibilities

Decisions and tradeoffs

1. **Modular monolith:** Easy to launch, inspect and debug within a senior design schedule. One process is a shared failure domain. Planned refactoring separates routing, validation, rules and repositories without adding distributed deployment.
2. **SQLite:** Transactions and parameters provide reliable local persistence without a database server. Synchronous database work can block the Node.js event loop; WAL does not eliminate that limitation.
3. **PowerShell polling:** Native Windows integration avoids another agent dependency. Polling introduces latency and can miss overwritten logs during outages.
4. **Deterministic rules:** Reproducible and explainable without training data. Coverage is narrow; a rule match requires investigation.
5. **Local boundary:** Loopback reduces remote exposure but does not authenticate local processes or protect against malware executing as the operator.
6. **Two second browser refresh:** Simple for one operator. Streaming and queues are deferred until measurements justify operational complexity.

Authentication and authorization

Existing: Ingestion and heartbeat require a randomly generated 32 byte secret encoded as hex, compared using a length check and timing safe comparison. Dashboard reads have no user login. Alert updates require `X-Network-Knight: dashboard` and global Host/Origin restrictions. That header helps resist browser cross origin mutation; it is not a user credential and can be sent by local programs.

Planned local controls: Explicit Windows ACLs for runtime files, an unelevated Node.js process, collector elevation only where necessary, and documented secret rotation. Code executing as the trusted operator remains a residual risk.

Conditional shared or remote extension: Before changing binding, add established identity provider authentication, server enforced viewer/analyst roles, administrator controls, privileged MFA, HTTPS, secure HttpOnly session cookies, CSRF protection, expiration and revocation. Collector credentials must never authorize operator actions. These are not existing components.

2 Database or Data Model Design

Existing schema

Timestamps are UTC ISO 8601 text. Application code supplies most constraints; existing tables lack foreign keys and severity/status CHECK constraints.

Table	Fields and types	Keys and constraints
events	id INTEGER; source TEXT; external_id TEXT nullable; timestamp TEXT; received_at TEXT; kind TEXT; severity TEXT; host TEXT; ip TEXT; username TEXT; message TEXT; details TEXT	id primary key; all remaining fields NOT NULL except external_id; details holds JSON text
alerts	id INTEGER; rule TEXT; title TEXT; severity TEXT; status TEXT default open; created_at TEXT; updated_at TEXT; group_key TEXT; evidence TEXT	id primary key; all remaining fields NOT NULL; evidence holds a JSON array of event IDs
collectors	id TEXT; last_seen TEXT; status TEXT; detail TEXT	id primary key; remaining fields NOT NULL; latest heartbeat replaces prior state

Existing indexes are partial unique `events(source,host,external_id)` when external_id is not null; correlation `events(kind,source,host,ip,username,timestamp)`; and `alerts(rule,group_key,created_at)`. Rules live in code. Events have no collector foreign key.

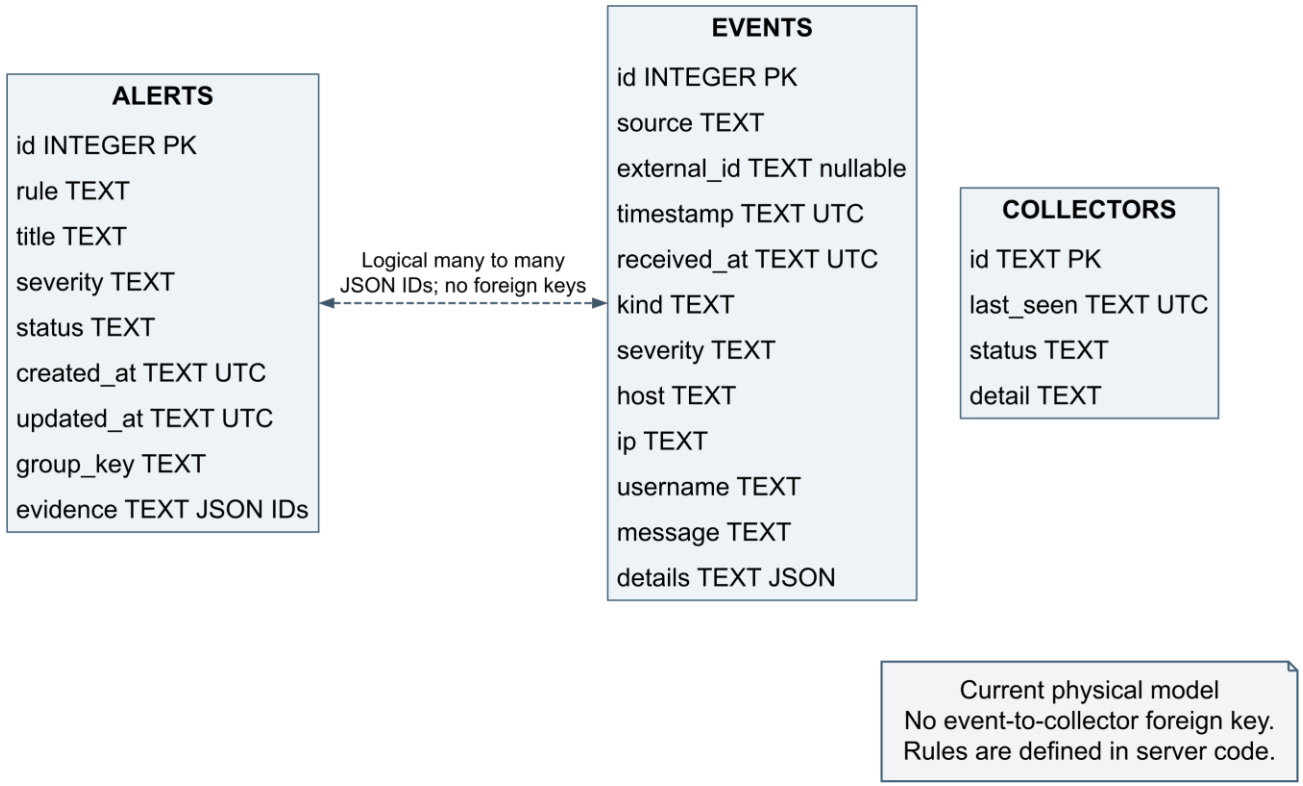


Figure 2 Existing physical schema and logical evidence relationship

An alert has at least one event when created; an event can support several alerts. The logical many to many relationship is not database enforced. Collectors are unconnected because the physical schema has no event relationship.

Planned evidence and action history

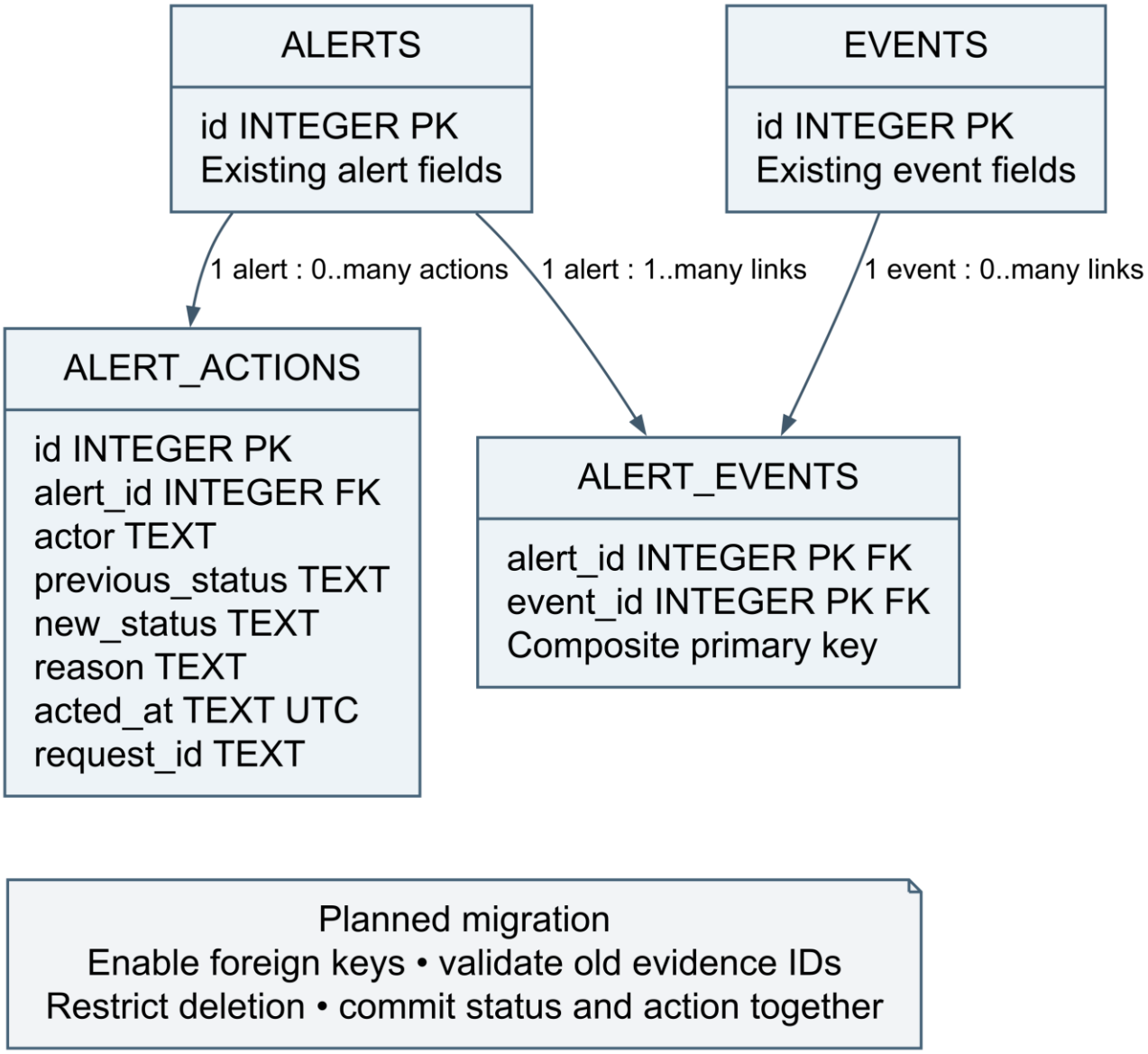


Figure 3 Planned evidence links and investigation action history

Enable `PRAGMA foreign_keys=ON` per connection; restrict deletion of linked evidence; add status, severity and JSON validity constraints. Commit each transition and audit row together. Local actor information records the OS operator context; distinguishing people requires authenticated sessions.

Migration: consistent backup; create tables; validate every evidence array and reference; backfill unique pairs; compare counts; switch readers/writers; retain old column through rollback validation. Missing references need review. No migration is executed by this report.

Data lifecycle

Validate complete batches before a transaction; insert events and derived alerts atomically. Stable Windows identity combines record ID and timestamp, with source and host in the unique index. Retries do not double count; a stolen key can still fabricate new identities.

Store event time and receipt time. Missing event time currently defaults to now; planned metadata should flag inferred times. Lab passwords are omitted, but Windows messages can expose accounts and paths. Minimize fields and sanitize exports.

Plan at least 90 days retention if adopting CIS safeguard 8.10, with unresolved evidence pinned and capacity/privacy approval. Current storage has no retention automation. Plan daily protected consistent backups and restore tests. Copying only a live SQLite file with active WAL is unsafe; use consistent backup support or stop the server cleanly before copying complete database state.

3 Technology Stack and Design Justification

Technology	Reason selected	Constraint
JavaScript	Shared server/browser language and inspectable rules	Explicit type and input validation needed
Node.js	Built in HTTP, crypto, filesystem and SQLite; no required npm install	Pin a supported compatible release; repository minimum 22.13, README prefers Node 24 LTS
node:http	Small allowlisted API and simple deployment	Team owns limits, routing and errors
SQLite and node:sqlite	Local transactions, parameters and low cost [5]	Synchronous work blocks event loop; limited write concurrency
HTML5 and CSS	Lightweight responsive interface and semantic controls	Direct accessibility testing required
Browser Fetch	Simple JSON queries and refresh	Polling cost and stale state handling
PowerShell and Get-WinEvent	Native Windows queries and XML extraction [6]	Windows only; channel permissions and audit policy
Loopback HTTP and JSON	Inspectable local interchange without cloud certificates	Unencrypted; trusted local boundary only
Node test runner and PowerShell fixtures	Rule, API, persistence and parser testing with few dependencies	Fixtures do not prove live collection
Git with excluded runtime data	Reviewable source without logs and keys	Ignore rules cannot remove secrets already committed

These choices fit the existing code without a React, FastAPI or cloud rewrite. Team proficiency is not measured; shared JavaScript reduces onboarding, while Windows administration and backend security testing still require assigned owners. Hosting and commercial subscriptions are not required; hardware, maintenance and protected backups still have costs.

4 Standards Cybersecurity Frameworks Benchmarks and Constraints

NIST CSF organizes risk outcomes, OWASP ASVS supplies application verification requirements, and CIS Controls guides operational safeguards. These are scoped mappings rather than compliance claims.

NIST Cybersecurity Framework 2 0

URL: https://www.nist.gov/cyberframework

Summary: A framework of cybersecurity outcomes organized into Govern, Identify, Protect, Detect, Respond and Recover. It is outcome oriented and does not prescribe one implementation [1].

Relationship: Monitoring and investigation directly support Detect and Respond. The application's own evidence also needs governance, protection and recovery.

Function	Application	Evidence or remaining work
Govern	Assign collection, retention, risk and deployment owners	Planned risk register and local use policy
Identify	Inventory host, channels, rules, data and dependencies	Architecture, source list and known gaps
Protect	Authenticate ingestion, validate inputs, restrict files and minimize data	Key checks exist; ACL and rotation verification planned
Detect	Six rules and freshness reporting	Threshold/grouping tests and source outage test
Respond	Evidence review and status workflow	Existing states; audit history and escalation runbook planned
Recover	Consistent backups and restore exercise	Planned measured recovery demonstration

Organization wide governance, institutional incident authority and enterprise recovery are outside this application's scope. Those outcomes require external processes and owners.

OWASP Application Security Verification Standard 5 0

URL: https://asvs.owasp.org/

Summary: Requirements for secure design and verification of web applications and services; the official site identifies 5.0.0 as its stable release [2].

Relationship: Network Knight accepts untrusted records, renders messages and changes investigation state through HTTP APIs.

Relevant area	Application	Verification or exclusions
Encoding and injection	SQL parameters, escaped output and static asset allowlist	Test hostile messages, markup and SQL characters
Validation and business logic	Atomic batch validation, body/time limits and status allowlist	Plan strict field types and valid bounded JSON
Authentication and authorization	Collector credential distinct from operator trust	OS boundary now; proper sessions and roles before remote use
Browser security	Host/Origin restrictions, mutation header, CSP and frame prevention	Browser testing; header does not authenticate a person
Data protection and logging	Omit lab passwords, protect files and audit decisions	ACLs, action history and retention planned
Communication/configuration	Loopback and supported runtime	TLS and secure sessions before network exposure

Maintain a version pinned applicability checklist containing requirement text, rationale, evidence and results. This report maps areas without inventing requirement IDs or asserting a verified ASVS level. Password storage requirements do not apply to current nonexistent operator password accounts; reassess if accounts are added. Remote communication requirements cannot be waived once deployment expands.

CIS Critical Security Controls v8 1

URLs: https://www.cisecurity.org/controls/v8-1 and https://www.cisecurity.org/controls/audit-log-management

Summary: Prioritized operational safeguards. Control 8 concerns collecting, alerting, reviewing and retaining audit logs [3].

Relationship: Collection and evidence preservation are central to Network Knight, but it cannot prove all host auditing is enabled.

Safeguard or control	Application	Status and limitation
8.1 Log management process	Define channels, review owner, retention and failure handling	Process approval planned
8.2 Collect audit logs	Windows channels and monitored endpoints	Existing; verify policy and permissions separately
8.3 Adequate storage	Measure volume and warn on disk pressure	Capacity measurement and warnings planned
8.4 Time synchronization	UTC event/receipt times and host clock review	UTC exists; skew reporting planned
8.9 Centralize logs	Query selected channels in one store	Partial single host scope; no enterprise aggregation
8.10 Retain logs	At least 90 days if adopting safeguard; pin unresolved evidence	No current automatic retention
8.11 Review logs	Weekly or more frequent documented review	Human process; dashboard access alone insufficient
Controls 3 and 6	Protect log data and restrict access	Collector key exists; ACL and deployment review planned

Safeguards have implementation group applicability; selecting some does not establish full group conformance. Enterprise network and multi asset coverage are outside current scope. A shorter approved privacy/capacity retention exception must be disclosed and cannot be claimed as 8.10 alignment.

Constraints

Windows is required for real collection. Rights and audit policy determine visibility; the project must not weaken organizational policy to gain access. Student hardware, disk and one backend constrain capacity. Logs identify users, hosts and paths; obtain host owner authorization and sanitize demonstrations. No FERPA, HIPAA or PCI DSS claim is established without deployment and data classification evidence.

5 Context Diagram

The boundary contains the delivered browser UI, backend, collector, rules, database and runtime files. The human operator, Windows Event Log service and lab client are external entities. Windows is external as a source even on the same workstation. Internal stores are not context entities.

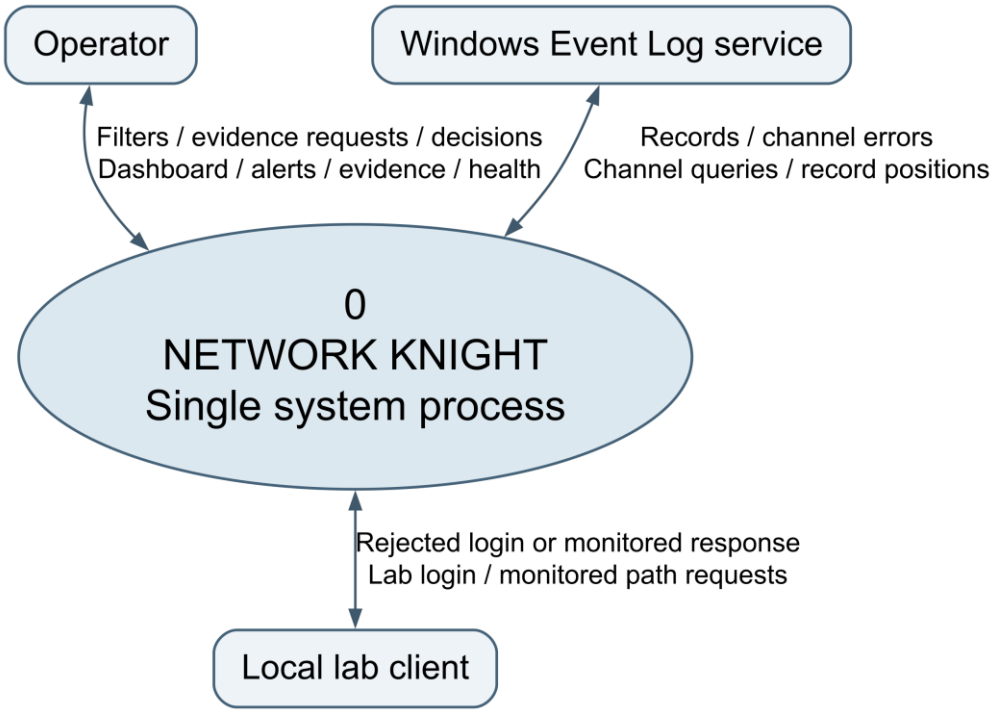


Figure 4 Level 0 context diagram

No external identity provider, notification service or cloud database exists in the current context. Remote access would change it.

6 Data Flow Diagram

Processes transform information; D1 to D4 are stores. D1, D2 and D3 are logical stores in one SQLite file. P2 invokes P3 within the ingest transaction.

External inputs and outputs balance the context diagram. The local HTTP path enters P2 as a server generated event; it is not an unauthenticated collector upload.

Ingestion and failure handling

1. Poll three channels every five seconds, at most 200 matching records per channel in oldest first order. First run starts at the current end; historical records are not automatically imported.
2. Parse XML into source, kind, host, account, available IP, UTC time and provider context. Missing values remain unknown.
3. Submit JSON with bearer credential. Validate Host and Origin where present, route, key, required fields, timestamp, batch and body limits. Each API batch has 1 to 500 events and at most 1 MiB.
4. Normalize and sort within the batch; begin transaction; deduplicate; insert new events and evaluate rules; commit.
5. Return accepted/duplicate counts. Advance a channel cursor only after acceptance. A lost reply followed by retry does not double count committed stable identities.
6. Persist cursors using temporary file replacement. Send heartbeat; unreadable channels are degraded. UI declares offline after 30 seconds without a fresh heartbeat.

Heartbeat proves recent communication, not complete collection. Audit configuration, overwritten records, outage duration and clock errors remain visibility risks.

Level 1 processes stores and balanced external flows

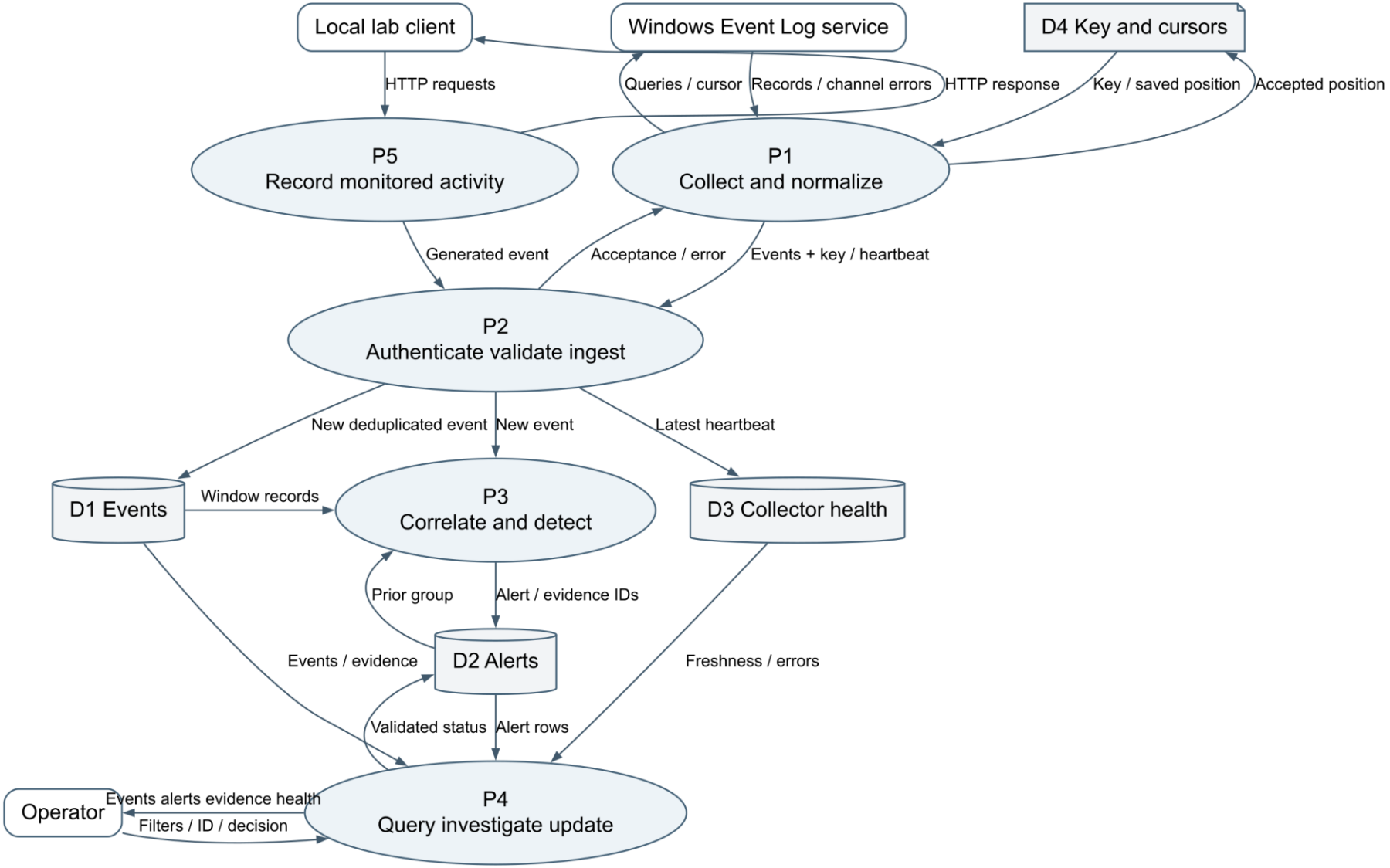


Figure 5 Level 1 processes stores and balanced external flows

Detection design

Rule	Condition and grouping	Severity and interpretation
AUTH-001	Five auth_failure records in inclusive 60 seconds; source, host, IP, username	High; review repeated failures
WEB-001	Five web_probe records in 60 seconds; source, host, IP	Medium; only this app's sensitive monitored paths
WIN-001	Security 1102 audit_cleared	Critical; possible evidence removal
WIN-002	Security 4720 account_created	Medium; verify authorization
WIN-003	Security 4697 or System 7045 service_installed	Medium; verify service provenance
WIN-004	Defender Operational 1116 malware_detected	High; inspect threat and remediation context

Security 4624, System 6005/6006 and Defender 1117 provide context. Repeated threshold matches within approximately one minute of alert creation extend evidence rather than generate repeated alerts. A resolved alert remains resolved within that suppression period even if evidence grows. Planned UI work must explain this and establish whether fresh evidence should reopen an investigation.

Sorting is within each batch. Very late events across batches are not fully re-correlated. Plan a bounded lateness policy and cross batch tests. An empty alert queue is not evidence that a host is safe.

API contracts

Method and path	Input or purpose	Response and access
GET /api/health	Liveness	Name, storage and uptime; local
POST /api/ingest	events array	Accepted/duplicate counts; bearer key
POST /api/heartbeat	id, status, detail	Latest health update; bearer key
GET /api/state	q and severity	Counts, newest 150 matching events, newest 100 alerts, rules, health
GET /api/alerts/:id	Numeric ID	Alert and available evidence; 404 if absent
PATCH /api/alerts/:id	open, acknowledged or resolved	Update; dashboard header and origin restrictions
POST /monitored/login	Lab username	Always 401; password is not stored

Plan request IDs, stable error codes, generic client errors, diagnostic logs, cursor pagination, rate limits and status reasons. Current search filters events, not the alert list; explain this scope. The 15 second request timeout does not guarantee database query cancellation.

7 User Interface and Experience Design

User flow

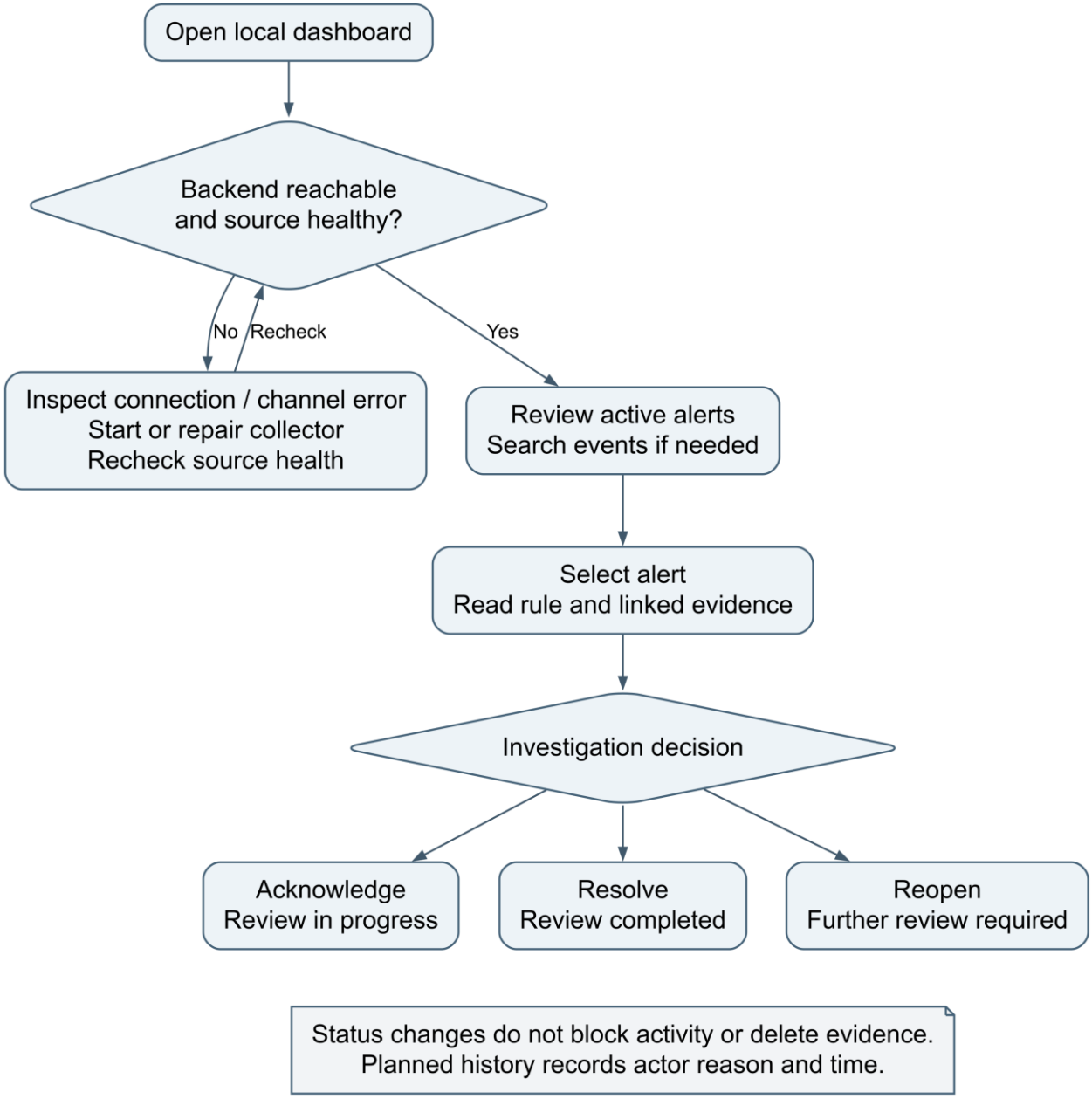


Figure 6 Operator investigation flow

Dashboard and evidence dialog wireframes

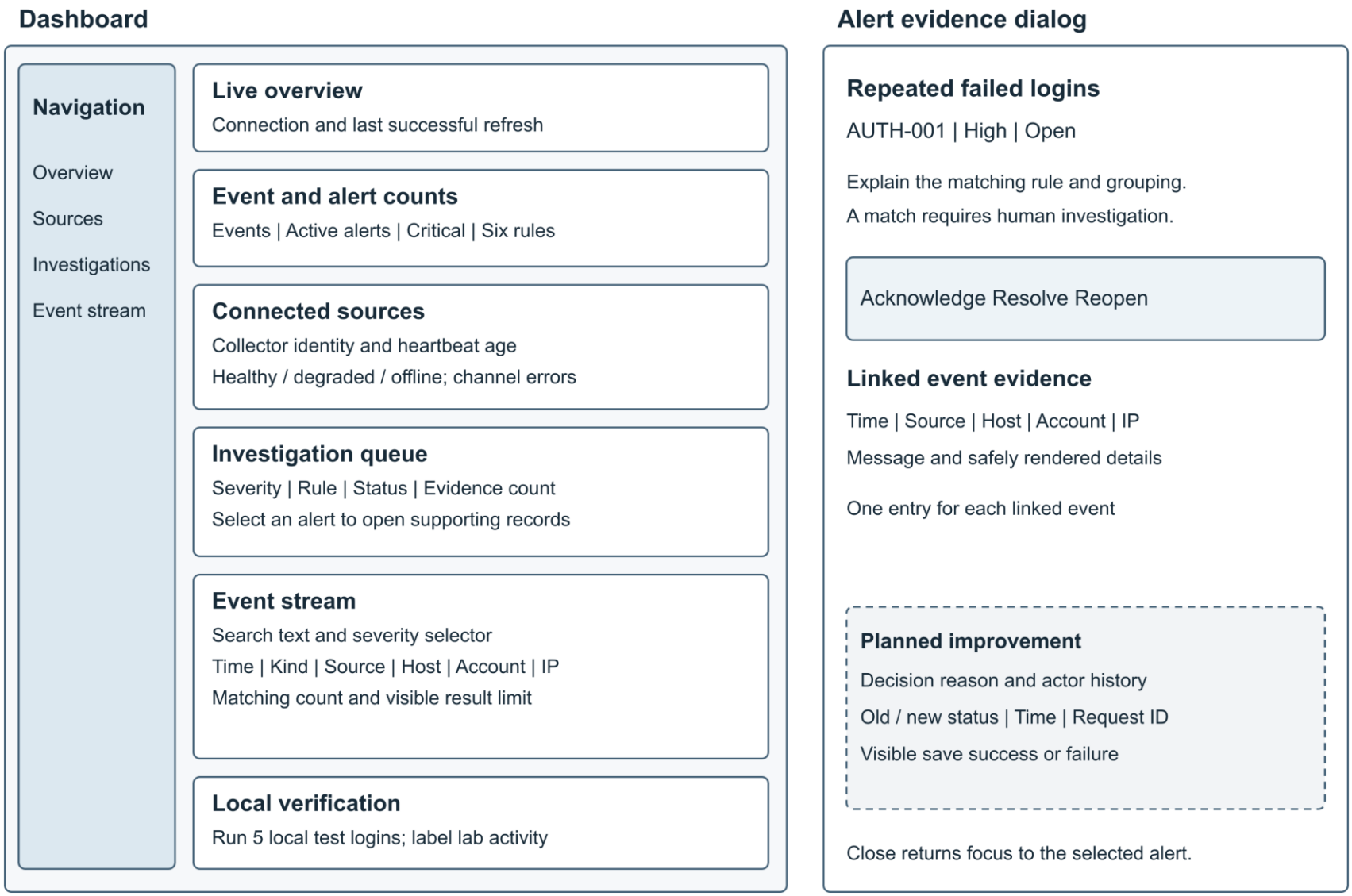


Figure 7 Dashboard and evidence dialog wireframes

Area	Content and behavior
Header	Connection, last successful update and stale warning
Summary	Stored events, active alerts, unresolved critical alerts and rule count
Sources	Collector identity, healthy/degraded/offline text, last heartbeat and channel errors
Queue	Rule title/ID, severity, status, evidence count and creation time
Stream	Account/host/IP/kind/message search, severity selection, matching count and display limit
Rows	Time, severity text, kind/message, source/host, account/IP
Lab	Explain five rejected requests; disable during run and show resulting alert

The evidence dialog displays title, rule, severity, state and each event's time, message, host, source, account, IP and escaped details. Existing actions are Acknowledge, Resolve, Reopen and Close. Add a complete trigger explanation and grouping context. Planned controls collect a reason and display actor/time/history. Use a version or updated time condition to reject stale concurrent changes. Failed updates must not imply success.

Require keyboard access, visible focus, labels, modal focus containment/restoration, contrast and severity words alongside color. Refresh must preserve typed text and focus. Retain stale results with a warning on failure. Stack panels on narrow screens and scroll wide tables horizontally. Existing labels and status roles cover part of this; full browser and screen reader behavior requires verification.

Network Knight Initial Demo

Design note: These screenshots illustrate Network Knight - Initial Demo, our primary interface design. Layout, features and integrations may change during development. They are examples of the proposed user experience.

The published demo identifies its data as synthetic and saved in this browser. Its BF-001, PE-001 and PS-001 rules are separate from the local implementation rules documented in Section 6.



Figure 8 Network Knight - Initial Demo | Security overview

The overview brings event volume, severity distribution, source categories and active alerts into one screen.

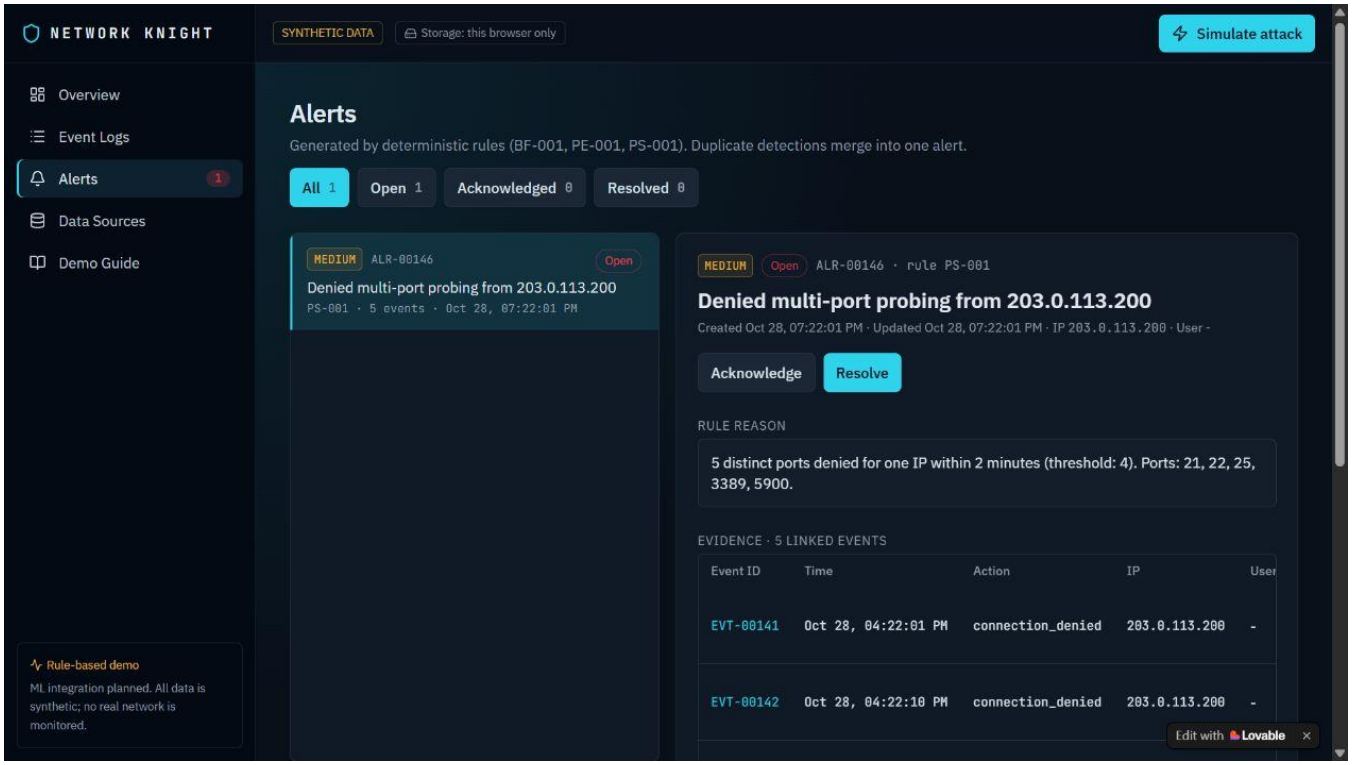


Figure 9 Network Knight - Initial Demo | Alert investigation and linked evidence

The investigation view exposes a rule explanation, supporting records and review actions.

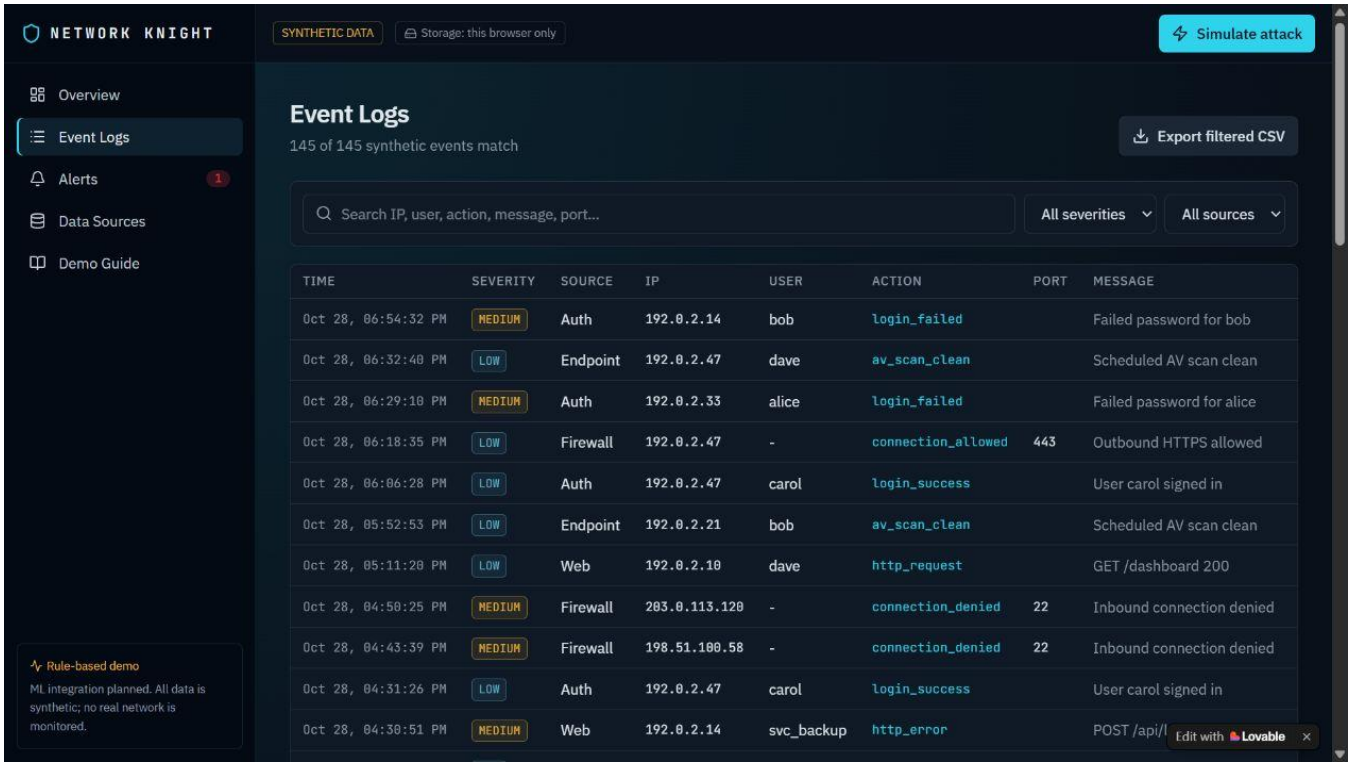


Figure 10 Network Knight - Initial Demo | Searchable event logs

8 Security and Risk Considerations

Threat Model Diagram

This model reuses DFD processes and stores. P4 includes the browser/server investigation path. Dashed arrows identify attacker activity. S, T, R, I, D and E refer to STRIDE [4]. Boundaries are logical controls, not network subnets.

Threat model with trust boundaries and STRIDE references

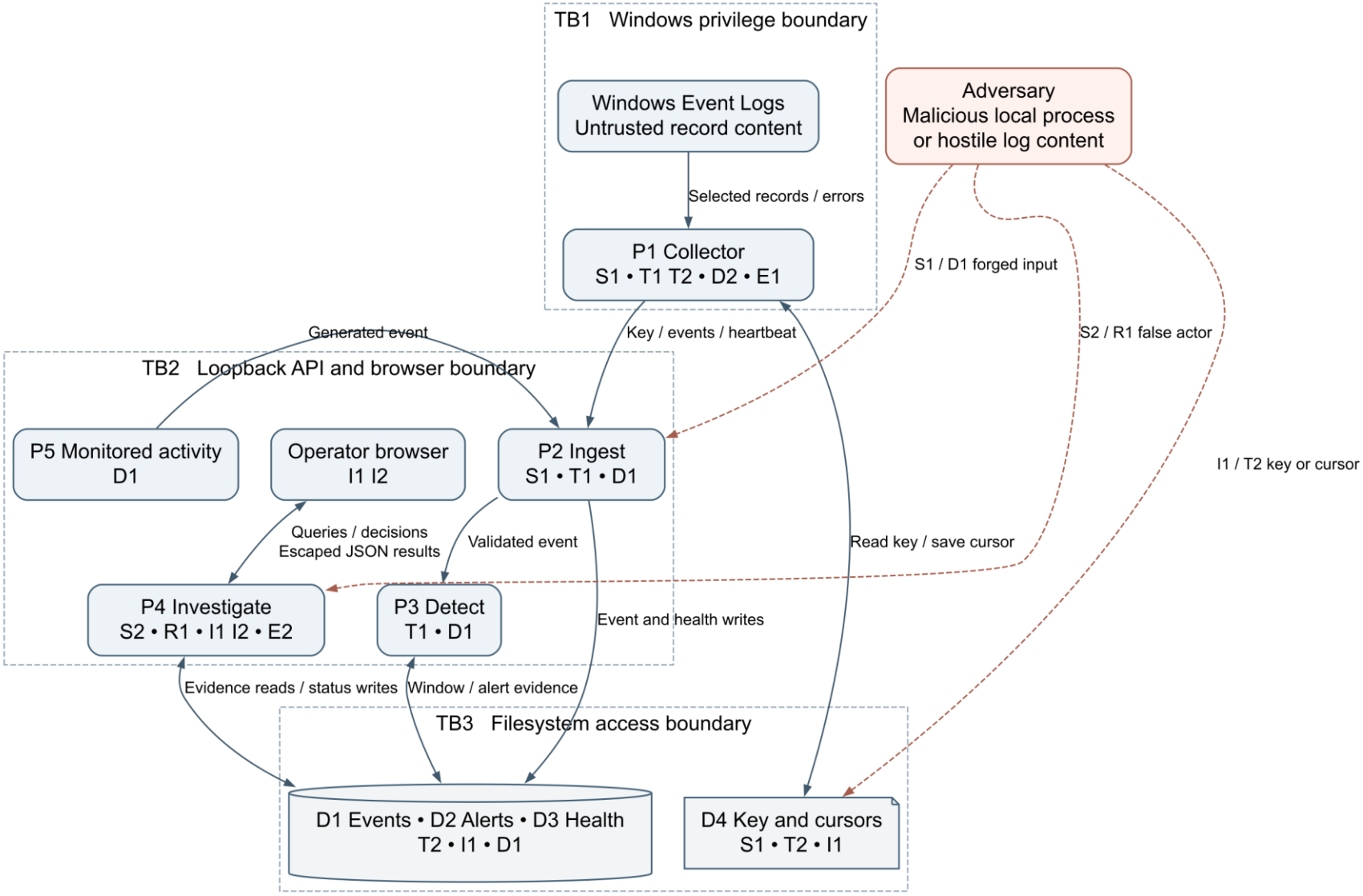


Figure 11 Threat model with trust boundaries and STRIDE references

TB1 separates privileged log access from ordinary execution; TB2 separates supplied data from trusted decisions; TB3 relies on OS file protection. Assets include evidence confidentiality/integrity, alert state, key secrecy, cursor correctness, availability and action history. Adversaries include another local account, operator context malware, hostile webpages and attackers producing malicious log content. A compromised administrator can cross all boundaries; host compromise defeats application only evidence guarantees.

STRIDE Analysis and Specific Mitigations

C is confidentiality, I integrity, A availability. Priority is a qualitative engineering judgment, not a measured exploit probability.

Threat	Components and scenario	Impact and priority	Specific controls and verification
S1 Spoofing	P1/P2/D4: stolen shared key fabricates events or heartbeats under arbitrary collector IDs	I high, A medium; high	Existing random key and timing safe check. Plan ACLs and rotation; individual credentials/source binding if expanded. Test invalid and revoked keys.
S2 Spoofing	P4: local code imitates dashboard header and changes alert state	I high, C medium; high	Explicit trusted local operator boundary. Add authenticated sessions/roles for stronger separation. Test that collector credential and header alone cannot authorize extended operator routes.
T1 Tampering	P2/P3/D1: manipulated timestamps, identities or fields alter detection grouping	I high; high	Existing limits, future time rejection, parameters, transaction and deduplication. Plan strict types, valid bounded JSON, clock skew and source binding. Test late/manipulated events.
T2 Tampering	D1/D2/D4: database or cursor changes erase evidence or skip logs	I high, A medium; high	Plan ACLs, protected backups, relational references and cursor checks. Protect integrity hashes separately if added. Test unauthorized file writes; colocated hashes cannot defeat an administrator.
R1 Repudiation	P4/D2: operator denies a status change; current store has no actor history	I high for accountability; high	Plan atomic append only application action rows with actor, old/new state, UTC time, reason and request ID. Protect audit copies separately for stronger assurance. Test exactly one history row per successful change.
I1 Information disclosure	Browser/P4/D1/backups: another user, package, screenshot or error reveals account, host and path data	C high; high	Existing loopback, no store responses and data exclusion. Plan ACLs, generic errors, protected backups and sanitized exports. Test another Windows identity. No existing encryption at rest claim.
I2 Information disclosure	Browser: log text executes script to read evidence or send actions	C high, I high; high	Existing escaping and CSP. Prefer text nodes where possible; test malicious message, details and username fields. Payload displays as text without execution.
D1 Denial of service	P2/P3/D1: repeated requests, nested details, large windows or disk growth block processing	A high, I medium from missed logs; high	Existing 1 MiB/500 event limits, truncation, request timeout and result caps. Plan rate/nesting/evidence bounds, indexes, retention and disk alerts. Measure saturation; row caps do not bound query work.
D2 Denial of service	P1/Windows: collector stopped, permissions revoked or channels overwritten	A high, I medium; high	Existing retry, cursor after acceptance, deduplication and heartbeat. Plan gap/backlog diagnostics and outage runbook. Test outages; warnings cannot restore overwritten events.

Threat	Components and scenario	Impact and priority	Specific controls and verification
E1 Elevation of privilege	P1/P2: backend flaw inherits admin rights if whole application is elevated	C/I/A high; high	Unelevated backend, minimum collector rights and isolated elevation. Never execute commands from event content. Verify process identities and ACLs.
E2 Elevation of privilege	Future P4 roles: viewer or collector reaches analyst/admin API	C high, I high; conditional high	Deny by default route/object authorization, separate credential scopes, server enforced roles and negative role tests. Hidden buttons are insufficient; current prototype has no roles.

Residual risks include privileged host compromise, operator context malware, incomplete auditing, shared key theft, false positives/negatives, overwritten records and late arrival limitations. Resolve through engineering or explicit scope acceptance.

Security Principles and Enforcing Controls

Principle	Enforcing control	Status
Least privilege	Loopback, unelevated backend, minimal collector rights and ACLs	Binding exists; deployment identity/ACL checks planned
Complete mediation	Every request validated; each upload/heartbeat authenticated; allowed statuses	Existing for these routes; operator roles conditional
Defense in depth	Input limits, SQL parameters, escaping, CSP, origin checks, OS protection, backup	Several layers exist; OS/recovery additions planned
Secure defaults and fail closed	Reject missing keys, invalid batches, foreign hosts/origins and unknown routes	Existing; stricter schema and generic errors planned
Minimal attack surface	Small route allowlist, no required npm packages, no DB listener	Existing; runtime patching remains operational work
Data minimization	Selected IDs, omit lab passwords, exclude runtime data	Partial existing; field allowlist/export planned
Integrity/accountability	Atomic ingest, deduplication, linked evidence and action history	First three exist; relational/action controls planned
Separation of duties	Ingestion credential distinct from operator access	Partial; sessions/roles needed for shared use
Availability/recovery	Freshness, retries, bounded work, storage warnings, backup	Health/retry exist; resource/recovery additions planned
Usable security	Visible gaps, explainable alerts and human review	Existing design; browser verification pending

Performance and Reliability Considerations

The nominal collector ceiling is 200 matching records per channel per five second cycle. Across three channels that is about 120 records per second only if work completes within the cycle; queries, sequential uploads and timeouts reduce actual capacity. Uploads are per channel, but large messages can exceed the byte cap.

Dashboard limits bound returned rows, not work. Wildcard searches, counts, evidence arrays and time windows can grow expensive. The current correlation index favors authentication; consider `(kind,source,host,ip,timestamp)` for web grouping after query plan and load analysis. Plan pagination and bounded evidence reads before scaling.

Example capacity scenario: 10 events/second at an assumed 2 KiB per record yields about 1.65 GiB daily, or 148 GiB over 90 days before indexes, WAL and backups. This is neither measured nor expected traffic. Measure size/rate before approving retention. Warn on pressure rather than silently discard required evidence.

Proposed target	Method	Acceptance criterion
UI latency	Documented workstation, 100000 stored events, 10 events/second, one operator, 15 minutes	p95 state response below 500 ms; record resources and plans
Visibility	Authorized Windows events; healthy no backlog run	p95 event to display within 10 seconds; report clock uncertainty
Replay safety	Resubmit accepted identities after lost reply simulation	No extra records or evidence counting
Atomicity	Invalid event mixed into valid batch	No partial event/alert commit
Rule boundaries	Four/five events, separate identities and outside window records	Exact expected alerts and evidence
Outage indication	Stop collector with server/browser active	Offline on first successful refresh after heartbeat age reaches 30 seconds
Persistence	Restart after acknowledged writes	Committed evidence and states remain
Recovery	Restore latest daily consistent backup in isolation	Proposed RPO at most 24 hours; RTO at most 30 minutes; demonstrate
Exhaustion	Authorized bursts and isolated low disk simulation	Controlled errors or warnings, no silent partial commits; record saturation

Record hardware, runtime, event mix, warmup, duration, p50/p95 latency, failures, accepted rate, backlog and peak storage. Use labeled benign and authorized fixtures for false positives and rule recall. Six successful rule tests do not establish production detection accuracy.

References

Official references consulted October 28 2026. Implementation grounding: local README, package.json, server.js, collector/start-collector.ps1, collector/windows-event.ps1, public/index.html and public/app.js. Runtime records and secret values were not needed.

1. NIST. Cybersecurity Framework 2.0. https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20 and https://www.nist.gov/cyberframework/faqs
2. OWASP. Application Security Verification Standard 5.0. https://asvs.owasp.org/ and https://owasp.org/projects/asvs
3. CIS. Controls v8.1 and Audit Log Management. https://www.cisecurity.org/controls/CIS-controls-navigator and https://www.cisecurity.org/controls/audit-log-management
4. Microsoft. Threat Modeling Tool threats and STRIDE. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
5. Node.js. SQLite API. https://nodejs.org/api/sqlite.html
6. Microsoft. Get-WinEvent. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.diagnostics/get-winevent

Architecture, priorities, mappings and benchmark targets are project engineering judgments. References explain technologies/frameworks; they do not certify this implementation.

7. scikit-learn. IsolationForest candidate documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html
8. Network Knight published initial demo. https://network-knightinitialdash.lovable.app/ Captured October 28 2026.

Appendix A Project Scope and Implementation Status

Network Knight is the team's AI Assisted Cybersecurity Log Monitoring and Threat Detection System. The current local build is an implementation baseline: collection, storage, rule evaluation and a working investigation interface. The longer term project concept includes AI assistance and broader operating system support. Those goals must not be presented as completed features.

What is implemented and what must still be built

Capability	Evidence in local project	Status
Local event dashboard	public/index.html, style.css, app.js	Implemented
Backend and persistent storage	server.js with Node.js HTTP and SQLite	Implemented
Six explainable rules	AUTH-001, WEB-001, WIN-001 through WIN-004 in server.js	Implemented
Windows channel collection	collector/start-collector.ps1 and windows-event.ps1	Implemented code; real permissions and collection still require demonstration
Evidence review and status changes	Alert detail and PATCH endpoints	Implemented; no operator audit history
AI or Isolation Forest	No model training, feature pipeline or scoring service in local build	Future work
Linux and macOS collection	No corresponding collectors in inspected build	Future work
React and FastAPI	Earlier discussed stack; current local UI uses plain JavaScript and Node.js	Architecture choice to reconcile before a rewrite
User login and role permissions	No operator accounts in existing local prototype	Required before shared or remote deployment
Retention, protected backups and decision history	Design proposals in this report	Planned

The Lovable dashboard and public website demonstrate a separate interface/deployment path. A public dashboard or embedded page does not establish a secure connection to the workstation collector. Do not assume cloud ingestion, AI inference, or local evidence synchronization exists without verifying the integration.

The small business and gas station use case motivates simple evidence review and lower operating cost. The current build does not integrate point of sale devices, payment systems, store cameras, routers or business accounts. Those sources need separate connectors, authorization and scope decisions.

Buildable scope recommendation

Complete and demonstrate the local vertical slice first: one workstation, selected logs, durable storage, explainable alerts and an operator decision. Keep that baseline stable while adding the minimum missing security controls. If AI remains a required final deliverable, implement it as a separate milestone with measurable evidence rather than describing the existing rule engine as AI.

There are two valid stack paths. Retaining Node.js and the existing browser UI preserves working code. Moving to React and FastAPI matches the earlier concept but requires a planned migration and renewed testing. The report

recommends retaining the baseline for the current milestone; it does not declare an unapproved rewrite or remove AI from the team's final requirements.

Original measurable requirements

Team requirement	How to test it	Relationship to other targets
At least 80 percent of test users open the dashboard, find an alert and view details without assistance	Give the same task to five representative users; at least four succeed; record time and errors	A usability target, not a claim of completed testing
Dashboard displays requested data within 3 seconds under normal team workload	Define event count, event rate and workstation; measure request to usable display	The later 500 ms API target is an optional stricter engineering target
At least 95 percent of supported events are processed and analyzed within 5 seconds of backend receipt	Measure received_at through committed event and derived alert availability; report p95	Excludes pre-receipt Windows polling; end to end visibility can take longer

These requirements preserve the team's existing goals. Establish the normal workload before claiming success. No production accuracy or capacity is inferred from a small lab demonstration.

Current verification evidence

On October 28 2026, all 13 Node.js dashboard and system tests passed again using the in process request transport and temporary test databases. They cover threshold behavior, identity separation, time windows, deduplication, authentication, atomic ingestion, Windows rule fixtures, evidence, state changes, persistence, source health, origin/Host restrictions and input limits. This is fresh test evidence for application logic; it does not prove live socket operation, native Windows collection, browser accessibility, or the proposed performance objectives.

Appendix B Step by Step Local Demonstration

This walkthrough uses the existing local application and its supplied verification tools. Complete Part 1 before attempting Windows collection. Record observed results; the expected outcomes below are not prefilled test results.

Preparation

Use an authorized Windows workstation, a compatible installed Node.js runtime, the network-knight project folder, a browser and PowerShell. Keep the Node.js server in a normal terminal; use a separate elevated PowerShell window only if channel access requires it. Do not share the data folder, collector key or unsanitized log screenshots.

Part 1 Demonstrate the application pipeline

1. **Open the project folder.** In File Explorer, open network-knight and confirm server.js, Start-Network-Knight.cmd, public, collector and tools are present. Keep this folder open so every later command runs from the correct location.
2. **Start the backend.** Double-click Start-Network-Knight.cmd and leave its terminal open. Expected: the startup message displays http://127.0.0.1:8787 and SQLite persistence. If Node.js is missing, install a compatible supported release before continuing; if the port is occupied, identify the existing process rather than launching repeated copies.
3. **Open the dashboard.** Enter http://127.0.0.1:8787/ in the browser. Expected: a connected backend and an update time. Windows may show Not connected because its separate collector has not started; this does not invalidate the local HTTP demonstration.
4. **Generate one identifiable test.** Under Test the complete pipeline, select Run 5 local test logins once and wait for completion. Expected: five rejected HTTP requests, five stored events and an AUTH-001 alert linked to those events. The endpoint intentionally has no valid accounts; the rejections are expected.
5. **Inspect the evidence.** Open the resulting alert and verify its rule ID, high severity, lab username, source and five evidence records. Explain the five-events-in-60-seconds rule. Record the alert ID and unique lab username; use them to locate the same investigation later.
6. **Demonstrate investigation state.** Select Acknowledge, verify the displayed state, then Resolve and verify it again. Explain that these actions record the current investigation status; they neither block a source nor delete the five events. Reopen only if demonstrating that action is useful.
7. **Check restart persistence.** Stop the backend with Ctrl+C, restart it with the launcher, refresh the browser and locate the saved alert. Expected: the evidence and last saved status remain. Record the observed result. A restart proves local persistence, not a backup restore.

Optional command-line equivalent: With the backend running, open a second normal terminal in network-knight and run node tools/verify-flow.js. Expected: a PASS message tracing five rejected HTTP logins to five stored events, an AUTH-001 alert and five evidence records. This creates a new test username; it is a separate run from the browser button.

Part 2 Demonstrate Windows collection

8. **Start the collector separately.** In a PowerShell window opened in network-knight, run .\collector\start-collector.ps1. Use authorized elevation if Security log reading requires it. Expected: a Windows source appears in Connected sources with recent heartbeat and ok. A degraded status means at least one channel could not be read; inspect the stated channel rather than describing collection as fully healthy.
9. **Wait until collection is ready.** Confirm ok before generating the Windows test. First startup begins at the current end of each channel, so events created before the collector starts are not a valid live collection test.

10. **Generate the supplied Windows scenario.** In another terminal in network-knight, run .\tools\Test-Windows-Detection.ps1. It attempts five rejected logons with a new nonexistent account and prints its username. Existing Windows failure auditing must generate Security 4625; the script does not enable auditing or change policy.
11. **Trace the Windows records.** Allow about two normal polling intervals, then search the printed username in the dashboard. Expected, when the source records are produced and collected: five windows:Security events and an AUTH-001 alert with linked evidence. Compare the source with the earlier local-web test to show that these are different ingestion paths.
12. **Check collection health and shutdown.** Stop only the collector with Ctrl+C while keeping the backend and dashboard open. Expected: the source becomes offline on a successful refresh after its heartbeat is at least 30 seconds old. Existing evidence remains visible. Restart the collector if continuing the lab; otherwise stop the backend with Ctrl+C when finished.

Troubleshooting checkpoints

Observation	Check	Interpretation
Dashboard cannot connect	Server terminal, exact localhost URL and port	Backend reachability must be restored before testing rules
Windows source is degraded	Named channel error and authorized read permissions	Do not treat a heartbeat as proof every channel works
PowerShell refuses the script	Local or organization execution policy with instructor/administrator	Do not override managed policy; record the blocked step
No 4625 records after the Windows test	Existing failure auditing and the test's printed username	No source event means collection/detection cannot demonstrate that scenario
Records exist but no alert appears	Matching source/host/IP/account, five events within 60 seconds and receipt ordering	Inspect grouping and timing; preserve the evidence for diagnosis

Evidence to retain

Capture only the meaningful checkpoints: connected source health, one alert with its five linked records, the persisted status after restart, and the offline transition if required. Sanitize identities and host details before sharing. Record date, runtime version, alert ID, scenario and pass/fail observation; never include the collector key or runtime database in the submission.

Appendix C How to Explain Network Knight

Start with the problem

A workstation records many security events, but raw logs are difficult to review quickly. Network Knight brings selected events into one place and highlights specific patterns with evidence. The operator still decides whether the activity is legitimate, suspicious or resolved.

Explain the system in six steps

1. Windows writes events such as failed logons or service installations into its own logs.
2. Our PowerShell collector reads selected new records. A collector is a reader and sender; it does not create Windows evidence or detect every attack.
3. The backend checks the collector credential and event format, then converts the records into a consistent structure. This is normalization.
4. SQLite saves those events. The backend checks the rules and saves an alert with IDs of the supporting records.
5. The browser asks the backend for current events, alerts and collector health every two seconds.
6. A person opens the alert, inspects the evidence, and acknowledges, resolves or reopens it.

A concrete example

Suppose five failed logons for the same source, host, IP and account arrive within 60 seconds. AUTH-001 creates a high severity alert and references those five stored events. The browser displays the alert; selecting it retrieves the evidence. Acknowledge means someone has started reviewing it. Resolve changes investigation status; it does not block an IP, fix an account or delete evidence.

For a controlled application demonstration, use the provided lab button. It sends five real HTTP requests to an endpoint that always rejects logins. That proves the HTTP event to database to rule to alert path. It does not prove collection of Windows logs. Demonstrate Windows collection separately after permissions and failure auditing are verified.

What the diagrams mean

Diagram	What to say
Architecture	These are the components we run and how they communicate. The backend is where validation, storage access and detection occur.
Context diagram	The entire application is one process. The operator, Windows logs and lab client are outside entities exchanging information with it.
Level 1 DFD	We expand the system into collection, ingestion, detection, investigation and lab recording. Stores identify where information rests.
Existing ERD	Events store observations, alerts store detections, and collectors store health. Alert evidence currently uses JSON IDs.
Planned ERD	A join table enforces evidence relationships and an action table records investigation decisions.
UI wireframe	This is where users see source health, alerts, searches and evidence. It describes interface structure.
Threat model	The same flows cross privilege, API/browser and filesystem boundaries where we apply security controls.

Explain the frameworks without memorizing them

NIST CSF helps us organize risk work: govern responsibilities, identify assets, protect data, detect suspicious events, respond through investigation and recover evidence. OWASP ASVS helps us verify the application itself, including input validation, safe output, credentials and access checks. CIS Controls helps us operate logging responsibly through collection, review, time accuracy, storage and retention. We map selected relevant practices; we are not claiming certification.

Explain STRIDE with this project

Category	Simple project example	Response
Spoofing	Someone pretends to be our collector	Credential validation, protected keys and planned rotation
Tampering	Someone edits event evidence or cursor position	Restricted file access, transactions and protected backups
Repudiation	Someone denies resolving an alert	Planned actor and transition history
Information disclosure	Logs or keys are exposed to another person	Local access boundary, ACLs, minimized fields and sanitized exports
Denial of service	Too much input fills disk or stalls detection	Limits now; rate controls, storage warnings and retention planned
Elevation of privilege	A backend flaw runs with administrator rights	Keep backend unelevated and restrict collector privilege

How to describe the AI honestly

The current build uses rules and has no trained model. If the final project requires AI assistance, a candidate is an anomaly scoring component such as Isolation Forest. This would supplement the rule evidence, not turn every anomaly into a confirmed attack.

A buildable AI milestone needs a defined event feature schema, a labeled or carefully reviewed baseline dataset, a training procedure, a versioned model, a scoring interface, score/threshold storage and a comparison against rules alone. Example candidate features are failed logon count per account/time window, distinct source count and event rate; these are proposed features, not existing implementation.

Use separate time periods or hosts for training and evaluation to reduce leakage. Record false positives on benign activity and recall on authorized scenarios. Preserve raw evidence and model version with each scored result, and show score explanations without claiming a probability of compromise. Keep scoring asynchronous or bounded so a model failure does not stop basic logging.

Python, scikit-learn and a FastAPI scoring service are possible implementation choices if the team retains that earlier direction. They remain a proposal until the team approves scope, integration, dependencies and evaluation criteria. Cross-platform collectors likewise need their own adapters and tests.

Suggested division of work

Workstream	Concrete responsibility	Integration handoff
Collection and normalization	Windows permissions, channels, parser, cursors and source health	Agreed JSON event schema and reliable upload behavior

Workstream	Concrete responsibility	Integration handoff
Backend and data	Ingestion, SQLite, evidence relationships, action history and APIs	Documented routes and test fixtures
Interface and usability	Health display, alerts, evidence dialog, errors and accessibility	UI wired to real API responses
Security and validation	STRIDE controls, negative tests, benchmarks, backup/restore and final evidence	Reproducible acceptance checklist

For three members, combine security validation across all workstreams and assign one integration owner. Do not assign names without agreement. AI is a separate cross-cutting milestone once its scope is approved. Every member should be able to trace one event through the full pipeline.

Professor questions and accurate answers

Question	Answer
Is this a SIEM?	It is a small local monitoring and correlation prototype with some SIEM-like functions, not a full enterprise SIEM.
Is it really AI right now?	No. Current detections use six explicit rules. AI assistance is a proposed final-stage component if retained in scope.
Where does data go?	The inspected local build stores events, alerts and health in SQLite on the workstation.
Does opening the dashboard start collection?	No. The PowerShell collector is a separate process.
What prevents fake events?	Ingestion requires a secret, but a stolen key or compromised host remains a risk.
Does resolving an alert stop an attack?	No. It changes the investigation state; automated blocking is not implemented.
Why this stack?	It matches existing working code and minimizes deployment dependencies and cost for a single-host milestone.
Are Linux and macOS supported?	Not by the inspected collectors. They require additional adapters and validation.
What is still missing?	Live environment validation, audit history, retention/backups and stronger deployment controls; AI and broader sources if final scope requires them.

A short speaking outline

Explain the problem and intended small business user. Identify the single-host architecture. Walk through one failed-logon event from collection to evidence. Show how the operator reviews an alert. State the protections and limitations. Finish with the next build milestone and the test that will prove it works.

The strongest explanation is a demonstrated vertical slice followed by a precise account of what remains to build.

Appendix D Implementation and Verification Checklist

Order	Work	Completion evidence
1 Baseline	Pin runtime; verify backend and collector identities and permissions	API/parser tests and authorized Windows collection
2 Integrity	Evidence migration, strict JSON and action history	Copy migration, count/integrity checks, rollback exercise
3 Local security	ACLs, separated privilege, rotation and generic errors	Negative access tests and clean source packages
4 Reliability	Gap/backlog/skew indicators and lateness/suppression policy	Outage, replay, reset and late arrival tests
5 Resources	Limits, storage warnings, retention and backup	Load benchmark, pinned evidence test and restore
6 Usability	Keyboard, focus, labels, errors, search scope and stale state	Browser walkthrough
7 Review	Reconcile requirements, diagrams, threats and frameworks with code	Each implemented control has evidence; exceptions have owners

All 13 application tests passed on October 28 2026 using the in process transport and temporary databases. Live socket operation, Windows collection, accessibility and performance remain separate verification tasks.